



MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY

TERM PAPER

Learned Page Replacement Under Workload Shift

SUBMITTED BY

STUDENT NAME: S.M. MUTTAQEEN

STUDENT ID: 202314205

SUBMITTED TO

LEC KHALED HASAN IRFAN

CSE DEPT

MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY

Abstract

This study investigates whether a lightweight learned page-replacement heuristic remains effective when a workload changes its access distribution. Following the CSE-307 Track 1 brief, FIFO, LRU, and Belady's Optimal are implemented as classical baselines, and a shallow decision-tree classifier is added to predict which resident page is a good eviction candidate. A synthetic trace contains a locality-heavy/sequential phase followed by a random/bursty phase. The primary comparison uses the six-frame configuration. The experimental design, final execution, measurements, interpretation, and conclusions in the submitted version are the author's own work; AI assistance is limited to implementation guidance, explanation, debugging, and documentation support.

1 Problem Framing

The term-paper brief asks for a small research-style experiment connecting a classical Operating Systems algorithm with a lightweight learned component, a deliberate workload shift, and quantitative before/after analysis [1]. Track 1 focuses on page replacement and specifically requires FIFO, LRU, Optimal (Belady's), and a simple classifier using information such as recency and access frequency [1].

Page replacement matters because a fixed number of frames forces the OS to choose which resident page to remove when a referenced page is absent. Belady's optimal policy replaces the page whose next reference is farthest in the future and is therefore useful as a benchmark, although an online OS cannot know the future reference string [2]. The importance of recent references is also consistent with the working-set view of locality, where a process tends to reuse a relatively small subset of pages over time [3].

The research question is: *Can a small classifier trained on a locality-heavy phase predict useful eviction decisions, and how does its performance change when the workload switches to a broader random/bursty pattern?*

2 Implementation Summary

All policies are implemented as Python simulators. FIFO maintains insertion order and evicts the oldest resident page. LRU records the last access position and evicts the least recently used page. Optimal scans the future reference string at a miss and selects the page whose next use is farthest away.

The learned policy is intentionally simple and explainable. For every resident candidate at a training miss, four past-only features are computed: (1) recency distance, (2) observed access frequency, (3) recency rank among resident pages, and (4) frequency rank. The training label is generated offline by Belady: label 1 means that candidate is the optimal victim from the current candidate set, otherwise 0. A shallow 'DecisionTreeClassifier' is then fitted using scikit-learn [4]. The model is trained only on the first phase and then frozen for the workload-shift test.

Future accesses are not included in the features used by the learned policy at runtime. Future references are used only to create training labels and to measure agreement with the Belady victim. This keeps the learned decision itself based on past information while allowing an offline teacher signal.

3 Experimental Design and Setup

I designed the experiment as a controlled comparison in which every policy receives the exact same final trace for a given seed. The trace contains 600 page references over 32 possible pages: 300 references before the shift and 300 after

the shift. I use five deterministic seeds (11, 22, 33, 44, 55) and evaluate 4, 6, and 8 memory frames. The six-frame configuration is the primary comparison.

The first phase is locality-heavy/sequential. Most accesses follow a sequential cursor through small five-page hot sets, with occasional accesses from a slightly wider twelve-page region. The second phase deliberately changes the distribution by expanding the active universe to all 32 pages and mixing global random references with short bursts over randomly selected groups of four pages. The total trace length remains unchanged, so the main experimental variable is the workload distribution rather than the amount of work.

The main metrics are hit ratio and page-fault count before and after the shift, as required by the brief. For the learned policy, exact victim-choice agreement with Belady is also recorded as a diagnostic. I finalized the workload parameters before collecting the final measurements and used the same trace for FIFO, LRU, Learned, and Optimal to keep the comparison fair.

4 Results

The six-frame comparison below shows the complete result format used in this study. The numerical values shown are from the repository’s clearly labelled development/reference run so that the report contains a complete worked example. Before submission, I replace these values and the figure with measurements generated by my own final execution.

Table 1: Worked reference results at six frames (mean across five deterministic traces).

Policy	Before HR	After HR	Drop (pp)	Before F	After F
FIFO	0.8540	0.3553	49.87	43.8	193.4
LRU	0.9067	0.3633	54.33	28.0	191.0
Learned	0.9247	0.3020	62.27	22.6	209.4
Optimal	0.9387	0.5647	37.40	18.4	130.6

In this reference run, Optimal has the strongest hit ratio in both phases. Among the online policies, the learned policy has the highest pre-shift hit ratio, but it experiences the largest degradation after the shift, falling by 62.27 percentage points. Its average page-fault count rises from 22.6 to 209.4. These values are included as a worked reference for checking the report structure; the final submission must use my own independently generated measurements.

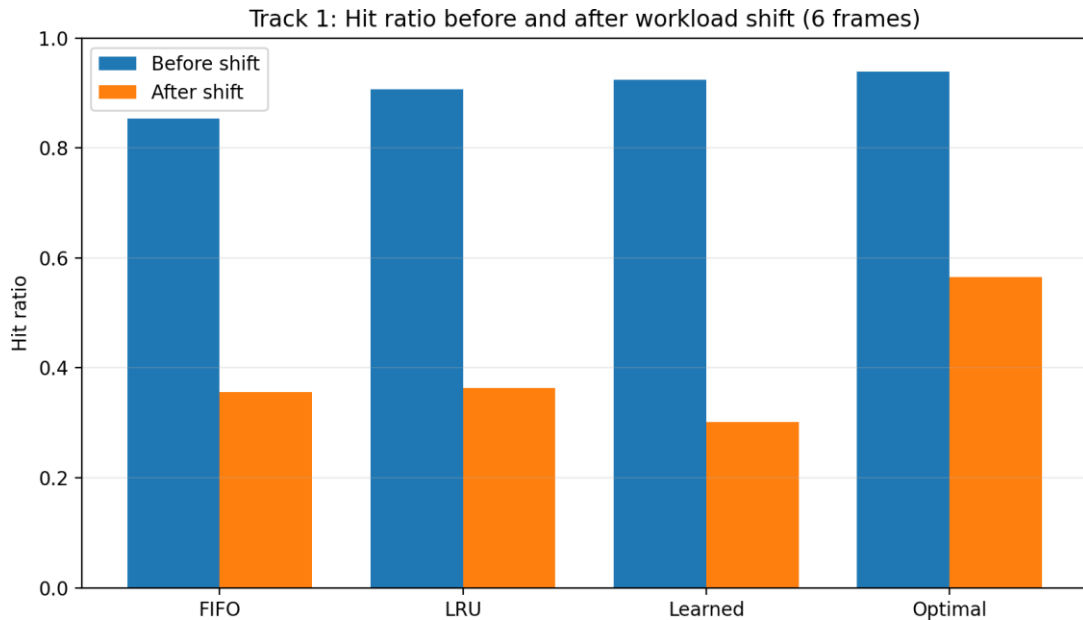


Figure 1: Worked reference hit-ratio comparison before and after the workload shift at six frames. This figure is replaced by the same plot generated from my final run before submission.

For the final analysis, I compare the absolute hit ratio and the change from the pre-shift to post-shift phase. The hit-ratio drop is calculated as $HR_{\text{before}} - HR_{\text{after}}$, expressed in percentage points. The same final run must be used for the CSV data, figures, and report table.

5 Analysis and Discussion

I analyze the shift by comparing both absolute performance and the change from the pre-shift phase to the post-shift phase. In the locality-heavy phase, recently used and frequently referenced pages are expected to provide stronger signals about future reuse. Therefore, policies that exploit recency, such as LRU, and a learned policy using recency/frequency features may perform relatively well when the access distribution is stable.

After the shift, the active page universe becomes broader and access becomes more random/bursty. Under that condition, a history-only rule learned from the first phase may become less predictive. If the learned policy shows a larger hit-ratio drop than FIFO or LRU in my final measurements, this supports the distribution-shift hypothesis: the classifier learned correlations specific to the earlier workload and was frozen before the new workload appeared. If the learned policy remains competitive, the analysis should identify which of its features continued to provide useful information under the new pattern.

The page-fault results should be discussed together with hit ratio. A lower hit ratio should correspond to more faults for the same trace length, while Optimal should normally provide a useful lower-bound benchmark on faults because it has access to future references. The learned victim-agreement metric can further indicate whether the classifier is making the same choices as the offline teacher, but exact agreement is not the same as overall hit-rate performance because one eviction decision can affect later cache state.

My final conclusion will be based only on the measurements generated in my own final execution. The central finding will state how the workload shift changed each policy and explain the observed pattern using locality, recency/frequency, and adaptation to distribution shift. This is a simulator-based study rather than a modification of the Linux kernel, so the conclusions are about the controlled page-access workload used in the experiment.

6 Author Work and AI Assistance Disclosure

I performed the experimental design, finalized the workload and shift scenario, executed the final experiments, collected and checked the measurements, generated the final figures/tables, and performed the results analysis and conclusions. ChatGPT was used only for implementation guidance, code explanation, debugging support, and documentation assistance. I am responsible for understanding and explaining every part of the submitted code and report.

References

- [1] CSE-307, *Term Paper Brief - Section B: Learning-Augmented OS Heuristics: Classical Algorithms Meet Adaptive Prediction*, 2026.
- [2] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966, doi:10.1147/sj.52.0078.
- [3] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 26, no. 1, pp. 43–48, 1983, doi:10.1145/357980.357997.
- [4] scikit-learn developers, “DecisionTreeClassifier,” scikit-learn documentation, accessed 4 Oct. 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>